

Central Processing Unit (CPU)

1. What is the CPU?

CPU = Central Processing Unit

The CPU is the main processing component of a computer.

A good definition for your notes:

The CPU is the component of a computer that executes program instructions and performs the processing required to transform input data into results.

In simple words:

CPU executes instructions and processes data.

2. Why is the CPU Important?

Suppose you write:

a = 10
b = 20
c = a + b

The computer has to:

1. Get the instructions.
2. Get the required data.
3. Perform the addition.
4. Store the result.

The CPU participates in these operations.

Conceptually:

Program + Data



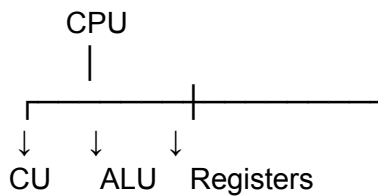
CPU



Result

3. Main Parts of the CPU

Your teacher's slide divides the CPU into three major parts:



Where:

CU

Control Unit

ALU

Arithmetic Logic Unit

Registers

Very fast temporary storage inside the CPU

Let's understand them one by one.

4. Control Unit — CU

The **Control Unit** controls and coordinates the activities of the computer.

It does not normally perform calculations itself.

Instead, it directs other components.

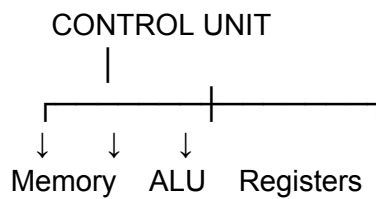
Think of the CU as a **coordinator**.

For example, if an instruction says:

ADD 10 + 20

the Control Unit coordinates the steps required to get the instruction and data and have the ALU perform the addition.

Simplified:



Key sentence

The Control Unit directs and coordinates the execution of instructions.

5. ALU — Arithmetic Logic Unit

ALU = Arithmetic Logic Unit

It performs:

Arithmetic operations

Addition +
Subtraction -
Multiplication ×
Division ÷

For example:

$$20 + 10 = 30$$

The ALU performs the arithmetic operation.

Logical operations

The ALU also performs logical/comparison operations.

For example:

$20 > 10$
 $20 < 10$
 $20 = 10$

It can also perform logical operations such as:

AND
OR

NOT

Therefore:

ALU performs arithmetic and logical operations on data.

6. Registers

This is where many students become confused.

A **register** is a very small, extremely fast storage location inside the CPU.

Think of registers as the CPU's **immediate working space**.

For example, while an operation is being performed, the CPU may need to temporarily hold:

Data

Instructions

Addresses

Intermediate results

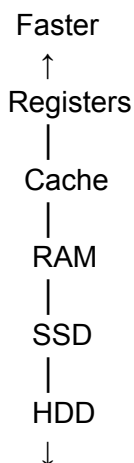
Registers hold such information.

7. Why Not Just Use RAM?

Excellent question.

Because registers are **inside the CPU** and are designed for extremely fast access.

A simplified hierarchy is:



Slower

Don't worry about memorizing the entire hierarchy yet.

The important point is:

Registers provide very fast temporary storage for CPU operations.

8. Important CPU Registers

Your teacher's slide includes these registers:

PC
CIR
MAR
MDR
AC

These are extremely important for understanding how the CPU executes instructions.

Let's take them one at a time.

9. PC — Program Counter

PC = Program Counter

The Program Counter stores the **address of the next instruction** that the CPU needs to fetch.

This is extremely important.

Imagine a program contains:

Instruction 1
Instruction 2
Instruction 3
Instruction 4

The CPU needs to know:

"Where is the next instruction?"

The **PC** keeps track of that location.

Conceptually:

PC

↓

Address of next instruction

Remember:

PC → Where is the next instruction?

10. MAR — Memory Address Register

MAR = Memory Address Register

The MAR holds the **address in memory that the CPU wants to access**.

For example:

MAR = 2050

This means, conceptually:

"CPU wants to access the data/instruction located at memory address 2050."

So:

MAR stores a memory address.

Remember:

MAR → Memory Address

The name itself gives you the answer.

11. MDR — Memory Data Register

MDR = Memory Data Register

The MDR temporarily holds the **data or instruction being transferred between the CPU and memory**.

For example:

Memory



MDR



CPU

Suppose the CPU requests something from memory.

The requested information can be transferred through the MDR.

Remember:

MDR → Data being transferred to/from memory.

12. CIR — Current Instruction Register

CIR = Current Instruction Register

The CIR holds the **instruction currently being executed/processed by the CPU.**

This is different from the PC.

PC:

Next instruction.

CIR:

Current instruction.

This distinction is **very important.**

Think:

PC



"What should I get next?"

CIR



"What instruction am I currently working on?"

13. AC — Accumulator

AC = Accumulator

The accumulator is a register used to hold **intermediate results and/or values involved in calculations**, particularly in simplified CPU models.

For example:

10 + 20

During processing, an intermediate or final result may be stored in the accumulator.

Conceptually:

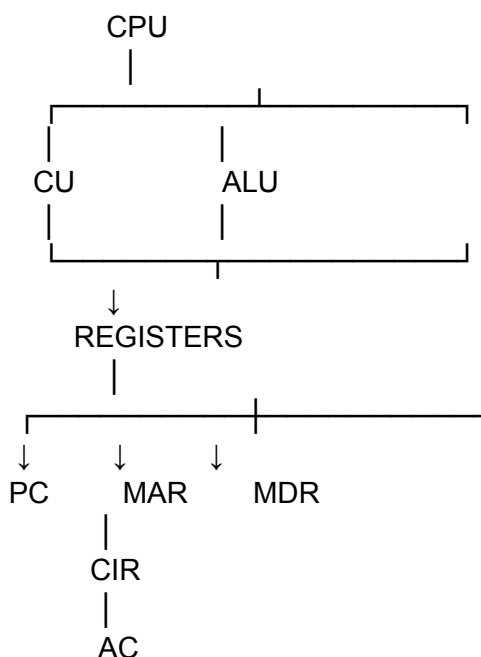
ALU
↓
AC
↓
Result

Remember:

AC → Holds intermediate/final results of processing.

14. Put the Registers Together

Now make this mental map:



More importantly, remember their jobs:

Register	Main purpose
PC	Address of next instruction
MAR	Address of memory location being accessed
MDR	Data/instruction transferred to/from memory
CIR	Current instruction
AC	Intermediate/result values

15. The Most Important Part — How Does CPU Execute an Instruction?

Now we're going to connect everything.

The CPU generally follows a process called the:

Fetch → Decode → Execute Cycle

This is fundamental.

```
FETCH
↓
DECODE
↓
EXECUTE
↓
FETCH
↓
DECODE
↓
EXECUTE
↓
...
```

The CPU repeats this cycle continuously while running programs.

16. Step 1 — FETCH

Suppose the PC contains:

PC = 1000

That means the next instruction is located at memory address 1000.

The CPU needs to fetch it.

Conceptually:

PC
↓
MAR

So the address from the PC is placed into the MAR.

MAR = 1000

The CPU now knows which memory location it wants.

17. Memory Sends the Instruction

Memory looks at address:

1000

and retrieves the instruction stored there.

The instruction is transferred to the:

MDR

So:

Memory
↓
MDR

Then the instruction is placed into:

CIR

So:

MDR



CIR

Now the CPU has the current instruction.

18. Step 2 — DECODE

The Control Unit examines the instruction in the CIR.

It determines:

What does this instruction mean, and what operations/components are required to execute it?

For example, suppose the instruction means:

ADD

The Control Unit recognizes that an addition operation is required.

It coordinates the necessary components.

19. Step 3 — EXECUTE

Now the CPU performs the operation.

If it is an addition:

10 + 20

the ALU performs the arithmetic operation.

10 + 20



ALU



30

The result may be temporarily stored in a register such as the accumulator.

ALU



AC = 30

20. Then What?

The CPU moves to the next instruction.

The PC is updated so that it points to the next instruction.

Then the CPU repeats:

FETCH



DECODE



EXECUTE



FETCH



DECODE



EXECUTE

This happens extremely rapidly.

That's one of the fundamental mechanisms through which programs execute.



Complete CPU Picture

Now you should be able to understand this:

